

КУРС «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ» · 1 СЕМЕСТР

06

Лекция 6
Конструкторы и RAII

Виды конструкторов · Member Initializer List и порядок инициализации · делегирующие конструкторы (C++11) · explicit · =default / =delete · деструкторы · RAII · время жизни объектов · Singleton · правило пяти

C++20 Конструкторы RAII

ПЛАН ЛЕКЦИИ

Десять глав сегодняшней лекции

01 Виды конструкторов

02 MIL и порядок инициализации

03 Делегирующие конструкторы (C++11)

04 Ключевое слово explicit

05 =default, =delete и таблица автогенерации

> **Сквозная идея:** конструктор определяет, как объект появляется на свет, деструктор — как он его покидает. Вместе они задают границы владения ресурсом.

06 Деструкторы и порядок уничтожения

07 Идиома RAII

08 Время жизни объектов

09 Перегрузка копирования и перемещения

10 Резюме и правило пяти

Четыре конструктора — четыре способа рождения

- **По умолчанию** — без аргументов; генерируется автоматически, пока не определён ни один конструктор
- **Параметризованный** — с аргументами, инициализирует поля
- **Копирующий** — `T(const T&)`, для ресурсов — глубокая копия
- **Конвертирующий** — один параметр, неявное преобразование типа

Uniform initialization

`Point p;`и `Point p2{};`— одно и то же. `{}`— единый синтаксис инициализации.

Пример
01_constructor_kinds



Поля инициализируются до тела — по порядку объявления

```
class Student {
    const int id_;                          // const — только через MIL
    std::string& group_ref_; // ссылка — только через MIL std::string name_;
    double gpa_;
public:
    Student(int id, std::string& group,
            const std::string& name, double gpa)

        : id_(id)                           // порядок в MIL не важен
        , group_ref_(group)
        , name_(name)
        , gpa_(gpa)
    {}
    // тело — для дополнительной логики
};
```

Пример
02_member_init_list



- **MIL** — единственный способ инициализировать const-поля, ссылки и объекты без default ctor
- поля инициализируются в порядке **объявления в классе**, а НЕ в порядке MIL
- несовпадение порядка — **частый источник UB**

Ловушка

```
class Danger {int b_;
    int a_;                             // b_ объявлен первым →
public:                                 // и инициализируется первым

    Danger(int x) : a_(x), b_(a_) {}                    // UB!
};
```

Конструктор вызывает конструктор — без дублирования

```
class Connection {std::string
        host_; int port_;
        bool ssl_;
public:
        Connection(const std::string& host,
                           int port, bool ssl)
                : host_(host), port_(port), ssl_(ssl) {
                // общая логика инициализации
        }
        Connection(const std::string& host, int port)

                : Connection(host, port, false) {}Connection(const      // ssl = false
        std::string& host)
                : Connection(host, 80, false) {}Connection()           // порт 80
                : Connection("localhost") {}
                                                                       // всё по умолч.
};
```

- один конструктор вызывает **другой конструктор того же класса**
- избегаем дублирования кода инициализации
- при делегировании нельзя инициализировать поля в MIL — только вызов

> Цепочка вызовов: ()→ ("localhost")→ (host, 80, false)→ полный конструктор.

Пример
03_delegating_constructors



МАИ · ООП · Лекция 6

Запрет неявных преобразований

```
class BigInt {std::vector<int> digits_;
public:
        explicit BigInt(int value) {
                while (value > 0) {digits_.push_back(value % 10);
                        value /= 10;
                }
        }
        explicit BigInt(const std::string& s) {
                for (char c : s) digits_.push_back(c - '0');
        }
};



// process(42);                 // Ошибка! explicit запрещает
process(BigInt(42));            // OK — явное создание
// BigInt b = 42; BigInt        // copy-initialization запрещена
b(42); BigInt b2{42};           // OK — direct initialization
                                // OK — direct-list-init
```

- explicitзапрещает **неявные преобразования** и **copy-initialization**
- остаются доступны direct initialization и явные вызовы

Правило

Конструктор с одним параметром делайте explicit, если неявная конверсия семантически неочевидна.

Пример
04_explicit



МАИ · ООП · Лекция 6

Перегрузка операций

Почему операция **std::cin >> file_text** имеет смысл?

- В C++ существуют механизмы, которые позволяют сопоставлять арифметический и другие операции, такие как побитовый сдвиг обычным функциям!
- Это позволяет лучше описывать типы.
- Мы можем описать не просто класс, но и операции с объектами этого класса.



Предупреждение

- Это механизм, при неумелом использовании которого можно полностью запутать код.
- Непонятный код – причина сложных ошибок!
- Перегруженные операции помогают определить «свойства» созданного вами класса, но не алгоритма работы с классами!

Перегрузка операций

Можно описать функции, для описания следующих операций:

+ - * / % ^ & | ~ !

= < > += -= *= /= %= ^= &=

|= << >> >>= <<= == != <= >= &&

|| ++ -- ->* , -> [] () new delete

Нельзя изменить приоритеты этих операций, равно как и синтаксические правила для выражений. Так, нельзя определить унарную операцию % , также как и бинарную операцию !.

```
type operator operator-symbol ( parameter-list )
```

Ключевое слово operator позволяет перегружать операции. Например:

- Перегрузка унарных операторов:
  - **ret-type operator** op **(** arg **)**

    где **ret-type** и op соответствуют описанию для функций-членов операторов, а arg — аргумент типа класса, с которым необходимо выполнить операцию.
- Перегрузка бинарных операторов
  - **ret-type operator** op**(arg1, arg2)**

    где *ret-type* и op — элементы, описанные для функций операторов членов, а arg1 и arg2 — аргументы. Хотя бы один из аргументов **должен принадлежать типу класса**.

Перегрузка оператора присваивания

Что показывает пример

Класс `TrackedBuffer` владеет динамическим буфером `int` и печатает каждый специальный метод. Это делает видимым, **что именно** вызывает компилятор:

- **копирующий конструктор** `TrackedBuffer(const TrackedBuffer&)` — глубокая копия: собственный буфер + поэлементное копирование;
- **копирующий оператор присваивания** `operator=(const TrackedBuffer&)` — реализован через *copy-and-swap* (сильная гарантия исключений и корректное самоприсваивание);
- **перемещающий конструктор** `TrackedBuffer(TrackedBuffer&&) noexcept` — забирает владение указателем, источник обнуляется;
- **перемещающий оператор присваивания** `operator=(TrackedBuffer&&) noexcept` — освобождает свой ресурс и забирает чужой (с проверкой самоперемещения);
- **деструктор** — освобождает `new[]` (для перемещённого объекта — безопасный `delete[] nullptr`).

Пример
05_copy_move_operators



Явный контроль специальных методов

Что показывает пример

- `Buffer` запрещает копирование (`= delete`) и реализует перемещение вручную.
- `Pod` явно просит стандартные методы через `= default`.
- Перемещение опустошает источник (`std::exchange`), поэтому `b1.size()` становится `0`.

Объявлено пользователем	Default ctor	Copy ctor	Copy assign	Move ctor	Move assign	Dtor
Ничего	да	да	да	да	да	да
Любой конструктор	нет	да	да	да	да	да
Copy ctor	нет	--	да	нет	нет	да
Copy assign	да	да	--	нет	нет	да
Move ctor	нет	del	del	--	нет	да
Move assign	да	del	del	нет	--	да
Destructor	да	да*	да*	нет	нет	--

* — генерируется, но deprecated.

Пример
06_default_delete



- =default— «оставь генерацию компилятору»
- =delete— «этой операции не будет»

Деструктор — точка возврата ресурсов

Что показывает пример

- Деструктор вызывается автоматически при выходе объекта из области видимости.
- Поля класса `C` (`a_`, `b_`) уничтожаются в **обратном** порядке объявления: сначала `b_`, потом `a_`.
- Локальные объекты уничтожаются в обратном порядке создания.

Пример
07_destructors_order



Захват в конструкторе, освобождение в деструкторе

Что показывает пример

- Класс `ScopedTimer` — RAII-обёртка над «ресурсом» (моментом старта): засекает время в конструкторе, печатает длительность в деструкторе.
- Деструктор вызывается автоматически при выходе из функции — как при обычном возврате (`measured`), так и при выходе через исключение (`measured_with_exception`).
- Копирование/перемещение запрещены: таймер привязан к своей области видимости.

- **RAII** — Resource Acquisition Is Initialization
- при исключении деструкторы всё равно вызываются — **раскрутка стека**
- стандартные обёртки: std::unique_ptr, std::lock_guard, std::fstream …

Пример
08_raii



Единственный экземпляр — через статический метод

Что показывает пример

- `Database` — единственный экземпляр, доступный через `getInstance()`.
- Локальная `static`-переменная (Magic Static) даёт ленивую, потокобезопасную (с C++11) инициализацию.
- Приватные конструктор и деструктор + удалённые копирование/перемещение гарантируют, что другого экземпляра создать нельзя.
- Две ссылки `a` и `b` указывают на один и тот же объект.

> **Предостережение:** Singleton часто считают анти-паттерном — глобальное состояние, сложно тестировать, скрывает зависимости. Используйте осознанно.

Пример
09_singleton



МАИ · ООП · Лекция 6

Конструкторы и деструкторы — основа управления ресурсами

```
// Оператор присваивания (copy-and-swap).
DynamicArray& operator=(DynamicArray other);

// Или два отдельных оператора
DynamicArray& operator=(const DynamicArray& other);
DynamicArray& operator=(DynamicArray&& other) noexcept;
```

1. Конструктор по умолчанию
2. Копирующий конструктор
3. Перемещающий конструктор
4. Оператор копирования
5. Оператор перемещения

\+ деструктор (но для RAII – не нужен)

Пример
10_rule_of_five



ИТОГИ

Ключевые концепции лекции

- **MIL** — инициализация полей до входа в тело конструктора.
- **Порядок инициализации** — по порядку объявления полей, а не по порядку в MIL.
- **Делегирование** — конструктор вызывает другой конструктор того же класса.
- **explicit** — запрет неявных преобразований и copy-initialization.
- **=default / =delete** — явный контроль генерации специальных методов.
- **RAII** — захват ресурса в конструкторе, освобождение в деструкторе.
- **Singleton** — единственный экземпляр с доступом через статический метод.

> **Связка с практикой:** задание DynamicArray объединяет все концепции — MIL, делегирование, explicit, глубокую копию, деструктор и =delete для перемещающих операций.

ФИНАЛ

Вопросы для самопроверки

- Когда компилятор перестаёт генерировать конструктор по умолчанию?
- Почему MIL предпочтительнее присваивания в теле конструктора?
- В каком порядке инициализируются поля? Почему порядок в MIL не определяет реальный порядок?
- Что будет, если конструктор с одним параметром не explicit, и передать int в функцию, ожидающую MyClass?
- Какие специальные методы компилятор перестаёт генерировать при объявлении деструктора?
- Что гарантирует RAII при возникновении исключения?
- Почему Singleton считается анти-паттерном? Когда его использование оправдано?
- Что такое Magic Statics в C++11?

> Конструкторы и деструкторы — основа управления ресурсами. RAII — самый важный приём C++ для безопасного кода.